INSTITUTO TECNOLÓGICO UNIVERSITARIO

Tecnicatura Universitaria en Programación

Programación 1

Gestión de Datos de Países en Python: Filtros, Ordenamientos y Estadísticas con Consola de Colores

---

INTEGRANTES

Silva, Maia

Sosa, Micaela

Ortiz, Ariadna

Robledo, Luz

PROFESORES

Rigoni, Cinthia

Hualpa, Ramiro

Mendoza, 16 de octubre de 2026

Índice

1. Introducción **3**
1.1 Contexto y objetivos 3
1.2 Alcance del proyecto 3

2. Marco teórico **4**
2.1 Lenguaje Python y programación modular 4
2.2 Estructuras de datos: listas y diccionarios 4
2.3 Persistencia con archivos CSV 4
2.4 Validación de datos y manejo de excepciones 5
2.5 Búsqueda, filtrado y comprensión de listas 5
2.6 Algoritmos de ordenamiento: método burbuja 5
2.7 Estadísticas descriptivas con funciones integradas 5
2.8 Interfaz de consola y códigos de escape ANSI 6

3. Decisiones técnicas y arquitectura **7**
3.1 Arquitectura general del programa 7
3.2 Estructura de datos y formato del archivo 7
3.3 Decisiones de diseño 8
3.4 Diagramas de flujo 8
3.5 Capturas de ejecución 11

4. Dificultades y conclusiones **16**
4.1 Dificultades técnicas y resolución 16
4.2 Conclusiones y reflexiones finales del grupo 16

5. Bibliografía y webgrafía **18**

6. Enlaces del proyecto **19**

1. Introducción

1.1 Contexto y objetivos

El presente informe documenta el desarrollo de un sistema de consola escrito en Python para gestionar información de países: nombre, población, superficie y continente. El programa permite dar de alta nuevos países, actualizar sus datos numéricos, buscarlos, filtrarlos, ordenarlos y obtener estadísticas generales, almacenando la información de forma persistente en un archivo CSV.

El objetivo general del proyecto es integrar los contenidos de la materia Programación 1 en una aplicación completa y funcional. Como objetivos específicos se plantearon:

- Aplicar estructuras de datos (listas y diccionarios) para representar y procesar una colección de registros.
- Implementar persistencia de datos mediante la lectura y escritura de archivos CSV con el módulo estándar csv.
- Validar las entradas del usuario y manejar errores con excepciones para evitar interrupciones inesperadas.
- Desarrollar operaciones de búsqueda, filtrado, ordenamiento (algoritmo burbuja) y cálculo de estadísticas.
- Mejorar la experiencia de uso en consola mediante colores y formato tabular con códigos de escape ANSI.

1.2 Alcance del proyecto

La aplicación se ejecuta íntegramente en la terminal y utiliza únicamente módulos de la biblioteca estándar de Python (csv y os), por lo que no requiere instalar dependencias externas. El sistema contempla las operaciones de alta, modificación y consulta de países. Quedan fuera del alcance la eliminación de registros, la edición del continente de un país ya cargado y cualquier interfaz gráfica; estas posibilidades se retoman como mejoras futuras en el apartado de conclusiones.

2. Marco teórico

Este apartado presenta los conceptos de programación que fundamentan las decisiones tomadas en el proyecto, respaldados con la documentación oficial de Python, bibliografía especializada y material de clase.

2.1 Lenguaje Python y programación modular

Python es un lenguaje de alto nivel, interpretado y de propósito general que admite distintos paradigmas de programación. Su sintaxis clara y su extensa biblioteca estándar lo convierten en una herramienta habitual para la enseñanza de la programación (Downey, 2015). En este proyecto se adoptó un enfoque procedural y modular: el programa se descompone en funciones, cada una con una responsabilidad acotada, que se definen con la palabra clave def y se invocan desde un menú principal (Python Software Foundation [PSF], s. f.-d).

Las convenciones de nomenclatura (nombres de funciones en minúsculas con guiones bajos, constantes en mayúsculas) y el uso de cadenas de documentación (*docstrings*) siguen las recomendaciones de las guías de estilo PEP 8 y PEP 257 (van Rossum et al., 2001; Goodger & van Rossum, 2001). Asimismo, la estructura `if __name__ == "__main__":` permite que el archivo se ejecute como programa principal sin que el menú se inicie cuando el módulo es importado desde otro archivo.

2.2 Estructuras de datos: listas y diccionarios

Una lista es una secuencia mutable y ordenada de elementos, mientras que un diccionario es una colección que asocia claves con valores y permite acceder a cada valor por su clave. El proyecto representa cada país como un diccionario con las claves `nombre`, `poblacion`, `superficie` y `continente`, y el conjunto de países como una lista de diccionarios. Esta combinación conserva el orden de inserción, permite recorrer la colección con un bucle `for` y vuelve el código más legible que si se usaran posiciones numéricas.

Los diccionarios también se utilizan como contadores: para obtener la cantidad de países por continente se recorre la lista y se incrementa el valor asociado a cada continente mediante el método `get`, que devuelve un valor por defecto cuando la clave aún no existe.

2.3 Persistencia con archivos CSV

CSV (Comma-Separated Values) es un formato de texto plano en el que cada línea representa un registro y los campos se separan mediante comas. El módulo estándar `csv` ofrece las clases `DictReader` y `DictWriter`, que leen y escriben cada fila como un diccionario tomando la primera línea como encabezado.

Los archivos se abren con la función `open()` dentro de un bloque `with`, que garantiza su cierre incluso si ocurre un error durante la operación. El parámetro `newline=''` evita saltos de línea duplicados al escribir y `encoding='utf-8'` asegura la correcta representación de tildes y caracteres especiales. La función `os.path.exists` permite verificar si el archivo existe antes de leerlo. Dado que el CSV almacena todo como texto, los campos numéricos deben convertirse explícitamente con `int()` al cargarse.

2.4 Validación de datos y manejo de excepciones

Una excepción es un evento que interrumpe el flujo normal de un programa cuando se produce un error en tiempo de ejecución. Python permite capturarlas con la estructura try/except para responder de forma controlada. En el proyecto se capturan ValueError (cuando int() recibe un texto no numérico), KeyError (cuando falta una columna en el CSV) e IOError (cuando no es posible leer o escribir el archivo).

A lo largo del cursado, aprendimos que los datos siempre deben validarse antes de ser utilizados. Las funciones solicitar_entero_positivo y solicitar_texto combinan un bucle while True con try/except y repiten la consulta hasta obtener un dato válido, lo que evita que el programa se interrumpa o almacene información inconsistente.

2.5 Búsqueda, filtrado y comprensión de listas

La búsqueda por nombre se resuelve con el operador in, que verifica si una cadena está contenida en otra, junto con el método lower() para ignorar la diferencia entre mayúsculas y minúsculas. Para construir listas nuevas a partir de una condición se utilizan comprensiones de listas (*list comprehensions*), una sintaxis compacta equivalente a un bucle con condicional:

```
resultados = [p for p in lista_paises if criterio in p["nombre"].lower()]
```

Los filtros por rango emplean comparaciones encadenadas, como min_pob <= p["poblacion"] <= max_pob, que Python permite expresar directamente y que resultan más claras que combinar dos condiciones con and.

2.6 Algoritmos de ordenamiento: método burbuja

El ordenamiento burbuja (*bubble sort*) recorre repetidamente la lista comparando pares de elementos adyacentes y los intercambia si están en el orden incorrecto; después de cada pasada, el elemento mayor queda ubicado en su posición definitiva. Su complejidad temporal es $O(n^2)$ en el peor caso y su uso de memoria adicional es constante, por lo que no resulta eficiente para grandes volúmenes de datos, pero su simplicidad lo hace ideal con fines didácticos.

Python incluye las funciones sorted() y list.sort(), basadas en el algoritmo Timsort, que son la opción recomendada en aplicaciones reales. En este proyecto implementamos el método burbuja de forma manual, sobre una copia de la lista, para demostrar el funcionamiento del algoritmo. Su estructura general es la siguiente:

```
for i in range(n):
    for j in range(0, n - i - 1):
        if debe_intercambiarse(lista[j], lista[j + 1]):
            lista[j], lista[j + 1] = lista[j + 1], lista[j]
```

2.7 Estadísticas descriptivas con funciones integradas

Python provee funciones integradas que simplifican el cálculo de estadísticas: max() y min() devuelven el elemento mayor y menor de una colección y admiten el parámetro key, al que se le puede pasar una función lambda para indicar el campo por el cual comparar; sum() suma los valores de un iterable y len() devuelve la cantidad de elementos. El promedio se obtiene dividiendo la suma

de los valores por la cantidad de registros. Para mostrar los resultados se utiliza el mini-lenguaje de formato de cadenas, por ejemplo {valor:,.2f}, que agrega separadores de miles y limita a dos los decimales.

2.8 Interfaz de consola y códigos de escape ANSI

Las terminales modernas interpretan secuencias de escape definidas en el estándar ECMA-48 que permiten modificar el aspecto del texto (Ecma International, 1991). Las secuencias SGR (*Select Graphic Rendition*) tienen la forma ESC[<código>m: por ejemplo, 31 corresponde al color rojo, 32 al verde, 33 al amarillo, 34 al azul, 36 al cian, 1 a la negrita y 0 restablece el formato original. En Python el carácter de escape se escribe como \033.

El proyecto define estas secuencias como constantes (ROJO, VERDE, AMARILLO, AZUL, CIAN, NEGRITA y RESET) y las utiliza para diferenciar visualmente mensajes de error, de éxito, advertencias, títulos y encabezados de tabla. Además, la alineación de las columnas se logra con el lenguaje de formato: {valor:<20} alinea a la izquierda en un ancho de 20 caracteres.

3. Decisiones técnicas y arquitectura

3.1 Arquitectura general del programa

El sistema se organiza en un único archivo, Integrador_programación.py, estructurado en funciones que pueden agruparse en cuatro capas lógicas: persistencia de datos, entrada y presentación, operaciones del sistema y control del flujo mediante el menú principal. Para ejecutarlo se requiere Python 3 y una terminal con soporte de colores ANSI; el comando de ejecución es python Integrador_programación.py.

Tabla 1. Funciones del programa y capa a la que pertenecen

Función	Responsabilidad	Capa
inicializar_csv()	Crea paises.csv con cuatro países de ejemplo si el archivo no existe.	Persistencia
cargar_paises()	Lee el CSV, convierte los campos numéricos y devuelve una lista de diccionarios; omite filas con errores.	Persistencia
guardar_paises()	Escribe la lista completa en el CSV y devuelve True o False según el resultado.	Persistencia
mostrar_tabla()	Imprime una lista de países en formato tabular con colores y separadores de miles.	Presentación
solicitar_entero_positivo()	Pide un número entero mayor que cero y repite la consulta hasta que sea válido.	Entrada
solicitar_texto()	Pide un texto y verifica que no esté vacío.	Entrada
agregar_pais()	Da de alta un país nuevo evitando duplicados y guarda el cambio.	Operaciones
actualizar_pais()	Modifica población y superficie de un país existente.	Operaciones
buscar_pais()	Busca países por coincidencia parcial o exacta del nombre.	Operaciones
filtrar_paises()	Filtra por continente, rango de población o rango de superficie.	Operaciones
ordenar_paises()	Ordena por nombre, población o superficie, en forma ascendente o descendente.	Operaciones
mostrar_estadisticas()	Calcula mayor y menor población, promedios y cantidad de países por continente.	Operaciones
menu()	Carga los datos, muestra el menú en un bucle y deriva cada opción a su función.	Control

3.2 Estructura de datos y formato del archivo

Los datos se almacenan en el archivo paises.csv, con una fila de encabezado y un registro por país. Durante la ejecución se cargan en memoria como una lista de diccionarios y, ante cada alta o

actualización, la lista completa se vuelve a escribir en el archivo. Si el archivo no existe, el programa lo crea con los siguientes datos iniciales:

Tabla 2. Datos iniciales del archivo paises.csv

nombre	poblacion	superficie	continente
Argentina	45376763	2780400	América
Japón	125800000	377975	Asia
Brasil	213993437	8515767	América
Alemania	83149300	357022	Europa

3.3 Decisiones de diseño

- **Persistencia inmediata.** Cada modificación se guarda en el CSV en el momento en que se realiza, de modo que los datos en disco nunca quedan desactualizados si el programa se cierra de forma inesperada.
- **Comparaciones sin distinción de mayúsculas.** Las búsquedas, los filtros por continente y la detección de duplicados convierten los textos a minúsculas con lower(), y los espacios sobrantes se eliminan con strip().
- **Prevención de duplicados.** Antes de pedir los demás datos, agregar_pais() verifica que el nombre no exista, evitando que el usuario complete información que luego sería descartada.
- **Validación de números.** Población y superficie deben ser enteros mayores que cero; cualquier otra entrada genera un mensaje de error y se vuelve a solicitar el dato.
- **Lectura tolerante del CSV.** Si una fila tiene un formato incorrecto, cargar_paises() la omite y muestra una advertencia, en lugar de detener toda la carga.
- **Ordenamiento sobre una copia.** ordenar_paises() aplica el método burbuja a una copia de la lista, por lo que el orden de los datos originales y del archivo no se altera al consultar.
- **Constantes para los colores.** Las secuencias ANSI se definen una sola vez al inicio del archivo, lo que facilita modificar la paleta sin recorrer todo el código.
- **Edición acotada.** La opción de actualización modifica solo población y superficie, que son los datos que cambian con el tiempo; el nombre y el continente se consideran datos que no varían.

3.4 Diagramas de flujo

La Figura 1 muestra el flujo principal del programa: al iniciar se cargan los datos desde el CSV y se presenta el menú, que se repite hasta que el usuario elige la opción 7. Cualquier opción distinta de las siete previstas genera un mensaje de error y se vuelve al menú.

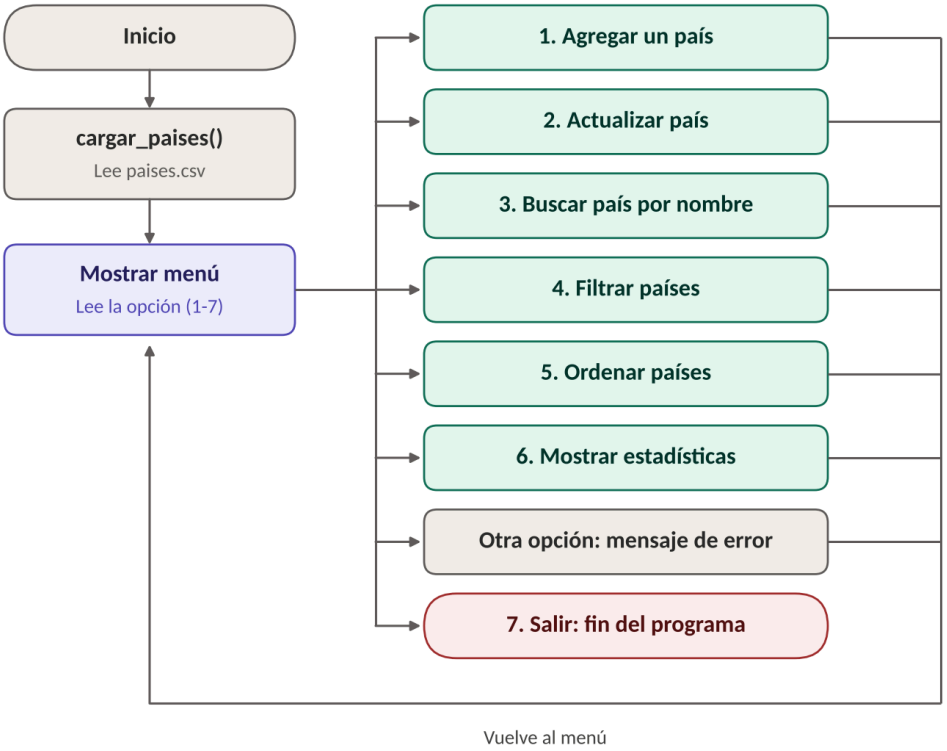


Figura 1. *Diagrama de flujo del programa principal y su menú de opciones.*

La Figura 2 detalla las operaciones de alta y actualización de países. En ambas se verifica primero el nombre ingresado (que no exista previamente en el alta, y que exista en la actualización) y recién entonces se solicitan los datos numéricos, validados como enteros positivos.

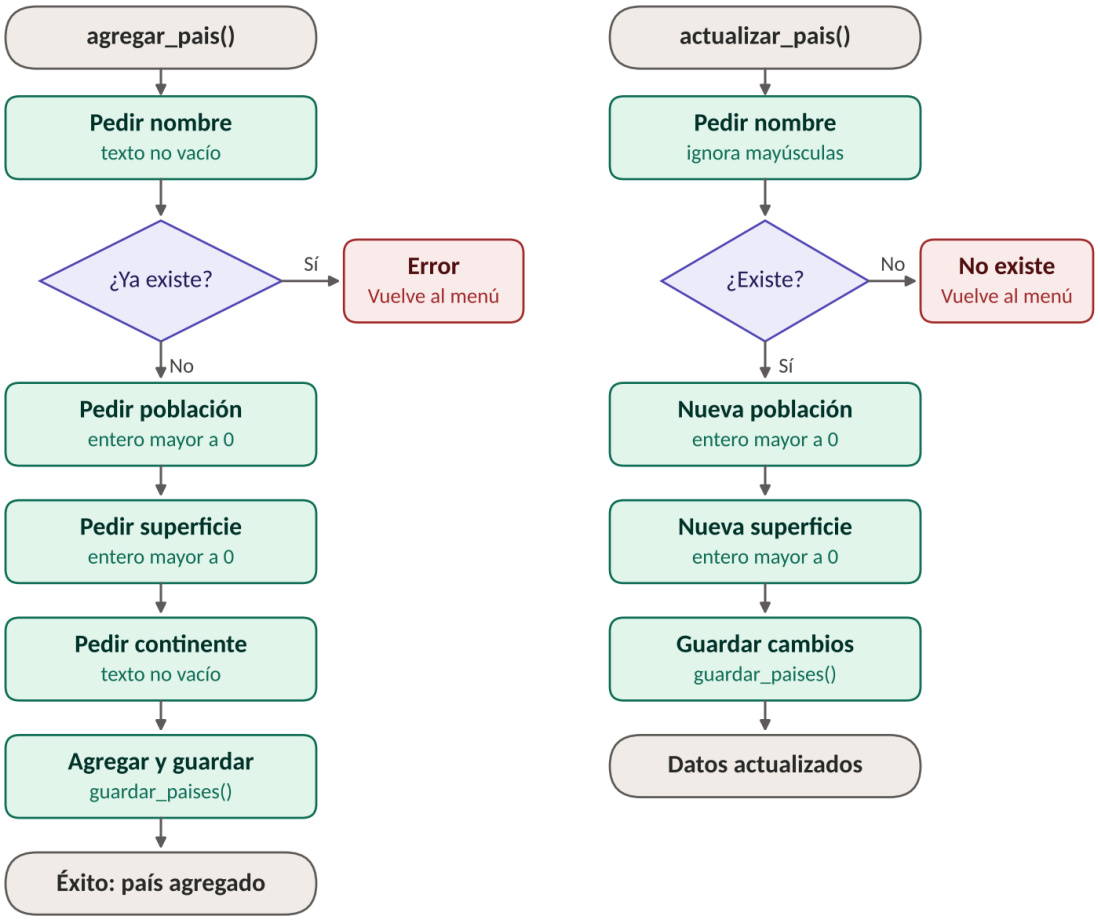


Figura 2. *Diagramas de flujo de las funciones agregar_pais() y actualizar_pais().*

La Figura 3 presenta el flujo de las funciones de filtrado y ordenamiento. Ambas validan la opción elegida en su submenú antes de procesar la lista y finalizan mostrando el resultado con mostrar_tabla(). Las funciones buscar_pais() y mostrar_estadisticas() no se diagraman por separado porque siguen un flujo lineal: la primera filtra por texto y muestra la tabla, y la segunda verifica que existan datos y calcula los indicadores.

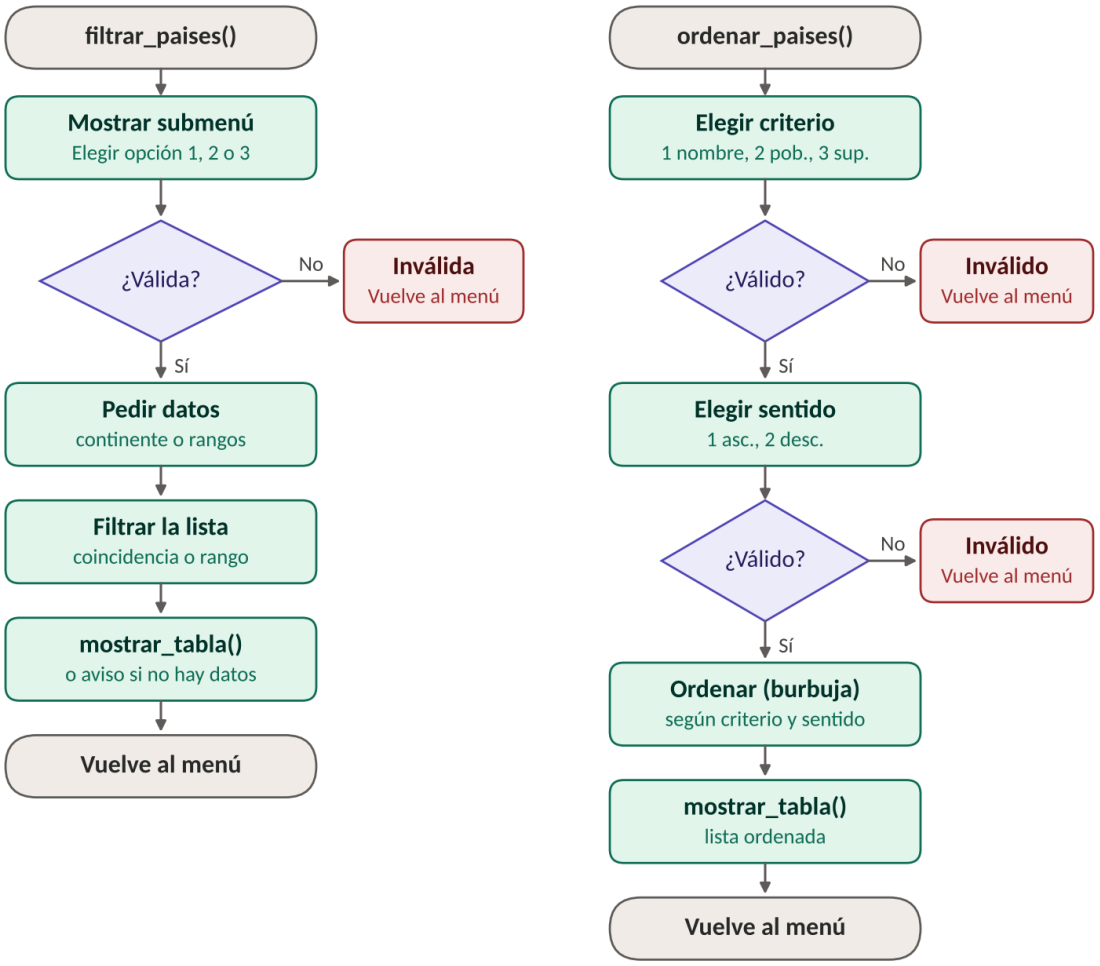


Figura 3. *Diagramas de flujo de las funciones filtrar_paises() y ordenar_paises().*

3.5 Capturas de ejecución

A continuación se presentan capturas de la ejecución del programa con los datos iniciales. En cada una se muestra la interacción completa, incluyendo los mensajes de error ante entradas inválidas.

```
========================================
   SISTEMA DE GESTIÓN DE PAÍSES (UTN)
========================================
1. Agregar un país
2. Actualizar población y superficie
3. Buscar un país por nombre
4. Filtrar países
5. Ordenar países
6. Mostrar estadísticas
7. Salir
----------------------------------------
Seleccione una opción (1-7): 9
Opción inválida. Intente nuevamente.
```

Figura 4. *Menú principal y mensaje de error ante una opción inexistente.*

La Figura 5 muestra el alta de un país. Primero se intenta registrar “Argentina”, que ya existe, y el sistema lo rechaza. Luego se carga “Chile” y se prueban entradas inválidas: un texto en lugar de un número, un valor negativo y un campo vacío. Cada error se informa en rojo y se vuelve a solicitar el dato hasta que es válido.

```
Seleccione una opción (1-7): 1

--- AGREGAR NUEVO PAÍS ---
Nombre del país: Argentina
Error: Este país ya se encuentra registrado.

Seleccione una opción (1-7): 1

--- AGREGAR NUEVO PAÍS ---
Nombre del país: Chile
Cantidad de población: abc
Entrada inválida. Debe ser un número entero.
Cantidad de población: -5
Por favor, ingrese un número mayor a 0.
Cantidad de población: 19493184
Superficie en km²: 756102
Continente:
Este campo no puede quedar vacío.
Continente: América
Éxito: 'Chile' ha sido agregado correctamente.
```

Figura 5. *Alta de un país con validación de duplicados y de entradas.*

La Figura 6 muestra la actualización de población y superficie de Brasil (el nombre se ingresa en minúsculas y el sistema lo reconoce) y el mensaje que se muestra al buscar un país inexistente.

```
Seleccione una opción (1-7): 2

--- ACTUALIZAR DATOS DE UN PAÍS ---
Ingrese el nombre del país a modificar: brasil
Modificando datos para Brasil:
Nueva población: 214326223
Nueva superficie en km²: 8515767
Éxito: Datos actualizados correctamente en el disco.

Seleccione una opción (1-7): 2

--- ACTUALIZAR DATOS DE UN PAÍS ---
Ingrese el nombre del país a modificar: Narnia
Error: El país buscado no existe.
```

Figura 6. *Actualización de datos de un país existente y de uno inexistente.*

La Figura 7 corresponde a la búsqueda por nombre con coincidencia parcial: al ingresar la letra “a” se listan todos los países cuyo nombre la contiene.

```
Seleccione una opción (1-7): 3

--- BUSCAR PAÍS ---
Ingrese el nombre (coincidencia parcial o exacta): a

Nombre               | Población       | Superficie (km²)   | Continente
-------------------------------------------------------------------------
Argentina            | 45,376,763      | 2,780,400          | América
Japón                | 125,800,000     | 377,975            | Asia
Brasil               | 214,326,223     | 8,515,767          | América
Alemania             | 83,149,300      | 357,022            | Europa
-------------------------------------------------------------------------
```

Figura 7. *Búsqueda parcial por nombre.*

La Figura 8 muestra dos filtros: por continente (“américa”, sin distinguir mayúsculas ni tildes de entrada) y por rango de población entre 50 y 150 millones de habitantes.

```
Seleccione una opción (1-7): 4

--- FILTRAR PAÍSES ---
1. Por Continente
2. Por Rango de Población
3. Por Rango de Superficie
Seleccione una opción de filtrado: 1
Ingrese el continente: américa

Nombre               | Población       | Superficie (km²)   | Continente
-------------------------------------------------------------------------
Argentina            | 45,376,763      | 2,780,400          | América
Brasil               | 214,326,223     | 8,515,767          | América
Chile                | 19,493,184      | 756,102            | América
-------------------------------------------------------------------------

Seleccione una opción (1-7): 4

--- FILTRAR PAÍSES ---
1. Por Continente
2. Por Rango de Población
3. Por Rango de Superficie
Seleccione una opción de filtrado: 2
Población mínima: 50000000
Población máxima: 150000000

Nombre               | Población       | Superficie (km²)   | Continente
-------------------------------------------------------------------------
Japón                | 125,800,000     | 377,975            | Asia
Alemania             | 83,149,300      | 357,022            | Europa
-------------------------------------------------------------------------
```

Figura 8. *Filtros por continente y por rango de población.*

La Figura 9 presenta el ordenamiento de los países por población en forma descendente y luego por nombre en forma ascendente, ambos mediante el método burbuja.

```
Seleccione una opción (1-7): 5

--- ORDENAR PAÍSES ---
1. Por Nombre
2. Por Población
3. Por Superficie
Seleccione el criterio de ordenación: 2
1. Ascendente
2. Descendente
Seleccione el sentido: 2

Nombre              | Población       | Superficie (km²)   | Continente
--------------------------------------------------------------------------------
Brasil              | 214,326,223     | 8,515,767          | América
Japón               | 125,800,000     | 377,975            | Asia
Alemania            | 83,149,300      | 357,022            | Europa
Argentina           | 45,376,763      | 2,780,400          | América
Chile               | 19,493,184      | 756,102            | América
--------------------------------------------------------------------------------

Seleccione una opción (1-7): 5

--- ORDENAR PAÍSES ---
1. Por Nombre
2. Por Población
3. Por Superficie
Seleccione el criterio de ordenación: 1
1. Ascendente
2. Descendente
Seleccione el sentido: 1

Nombre              | Población       | Superficie (km²)   | Continente
--------------------------------------------------------------------------------
Alemania            | 83,149,300      | 357,022            | Europa
Argentina           | 45,376,763      | 2,780,400          | América
Brasil              | 214,326,223     | 8,515,767          | América
Chile               | 19,493,184      | 756,102            | América
Japón               | 125,800,000     | 377,975            | Asia
--------------------------------------------------------------------------------
```

Figura 9. *Ordenamiento por población (descendente) y por nombre (ascendente).*

Por último, la Figura 10 expone las estadísticas generales calculadas sobre la lista de países cargada.

```
Seleccione una opción (1-7): 6

========================================
       ESTADÍSTICAS GENERALES
========================================
País con MAYOR población: Brasil (214,326,223 hab.)
País con MENOR población: Chile (19,493,184 hab.)
Promedio de población global: 97,629,094.00 habitantes
Promedio de superficie global: 2,557,453.20 km²

Cantidad de países por continente:
 - América         : 3
 - Asia            : 1
 - Europa          : 1
```

Figura 10. *Estadísticas generales: extremos, promedios y conteo por continente.*

4. Dificultades y conclusiones

4.1 Dificultades técnicas y resolución

Durante el desarrollo surgieron distintos obstáculos técnicos. La siguiente tabla resume los principales y la solución adoptada en cada caso.

Tabla 3. Dificultades encontradas y soluciones aplicadas

Dificultad	Solución adoptada
Los caracteres especiales (tildes, “Japón”, “América”, “km²”) se mostraban o guardaban de forma incorrecta.	Se especificó encoding='utf-8' en todas las aperturas del archivo CSV.
El programa se interrumpía cuando el usuario ingresaba texto en lugar de números, valores negativos o campos vacíos.	Se crearon las funciones solicitar_entero_positivo() y solicitar_texto(), que repiten la consulta con un bucle y capturan ValueError.
Era posible registrar el mismo país dos veces escribiéndolo con distintas mayúsculas (“argentina” y “Argentina”).	Se comparan los nombres convertidos a minúsculas con lower() antes de aceptar el alta.
El archivo CSV podía no existir o contener filas con formato incorrecto.	inicializar_csv() crea el archivo base y cargar_paises() omite las filas defectuosas informando una advertencia.
Las columnas de la tabla se desalineaban con números de muchas cifras.	Se usó el mini-lenguaje de formato con ancho fijo y separador de miles ({valor:<15,}).
Implementar el ordenamiento manual con campos de texto y numéricos, en ambos sentidos.	Se convierten los textos a minúsculas antes de comparar y la condición de intercambio depende del sentido elegido; se trabaja sobre una copia de la lista.
Los colores de la consola se aplicaban a las líneas siguientes.	Cada mensaje coloreado termina con la secuencia RESET, definida como constante.

4.2 Conclusiones y reflexiones finales del grupo

El desarrollo de este proyecto nos permitió integrar en una única aplicación los contenidos trabajados en Programación 1. Comprobamos que dividir el programa en funciones pequeñas, con una tarea clara cada una, facilita tanto la escritura como la prueba y la corrección del código, y que reutilizar funciones de validación evita repetir lógica en cada opción del menú.

Aprendimos además que validar las entradas y manejar las excepciones no es un detalle menor: una aplicación que se interrumpe ante un dato inesperado resulta poco confiable, aun cuando su lógica principal sea correcta. Trabajar con archivos CSV nos mostró la importancia de la persistencia y de cuidar aspectos como la codificación de caracteres y la consistencia entre los datos en memoria y los datos en disco. Implementar el ordenamiento burbuja de forma manual nos ayudó a comprender cómo funciona un algoritmo clásico y por qué, en la práctica, conviene recurrir a las herramientas que ofrece el lenguaje cuando se trabaja con grandes volúmenes de datos.

Como equipo, la experiencia reforzó la importancia de la comunicación y de acordar convenciones comunes (nombres, estructura y estilo del código) para integrar el trabajo de todas. Como líneas de mejora futura identificamos:

- Agregar la eliminación de países y la edición del continente.
- Permitir ordenamientos por más de un criterio y reemplazar el método burbuja por sorted() para comparar su rendimiento.
- Incorporar pruebas automáticas para verificar el comportamiento de cada función.
- Migrar el almacenamiento a una base de datos y ofrecer una interfaz gráfica o web.

5. Bibliografía y webgrafía

Universidad Tecnológica Nacional. *Material de Cátedra de Programación 1*

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). MIT Press.

Downey, A. B. (2015). *Think Python: How to think like a computer scientist* (2.ª ed.). O'Reilly Media. https://greenteapress.com/wp/think-python-2e/

Ecma International. (1991). *ECMA-48: Control functions for coded character sets* (5.ª ed.). https://ecma-international.org/publications-and-standards/standards/ecma-48/

Goodger, D., & van Rossum, G. (2001). *PEP 257 – Docstring conventions*. Python Software Foundation. https://peps.python.org/pep-0257/

Knuth, D. E. (1998). *The art of computer programming: Vol. 3. Sorting and searching* (2.ª ed.). Addison-Wesley.

Python Software Foundation. (s. f.-a). *Built-in functions*. Python 3 documentation. https://docs.python.org/3/library/functions.html

Python Software Foundation. (s. f.-b). *csv — CSV file reading and writing*. Python 3 documentation. https://docs.python.org/3/library/csv.html

Python Software Foundation. (s. f.-c). *Data structures*. The Python tutorial. https://docs.python.org/3/tutorial/datastructures.html

Python Software Foundation. (s. f.-d). *Defining functions*. The Python tutorial. https://docs.python.org/3/tutorial/controlflow.html#defining-functions

Python Software Foundation. (s. f.-e). *Errors and exceptions*. The Python tutorial. https://docs.python.org/3/tutorial/errors.html

Python Software Foundation. (s. f.-f). *Format specification mini-language*. Python 3 documentation. https://docs.python.org/3/library/string.html#format-specification-mini-language

Python Software Foundation. (s. f.-g). *Input and output*. The Python tutorial. https://docs.python.org/3/tutorial/inputoutput.html

Python Software Foundation. (s. f.-h). *os.path — Common pathname manipulations*. Python 3 documentation. https://docs.python.org/3/library/os.path.html

Python Software Foundation. (s. f.-i). *Sorting techniques*. Python 3 documentation. https://docs.python.org/3/howto/sorting.html

Shafranovich, Y. (2005). *RFC 4180: Common format and MIME type for comma-separated values (CSV) files*. IETF. https://www.rfc-editor.org/rfc/rfc4180

van Rossum, G., Warsaw, B., & Coghlan, A. (2001). *PEP 8 – Style guide for Python code*. Python Software Foundation. https://peps.python.org/pep-0008/

Wikipedia. (s. f.). *ANSI escape code*. En *Wikipedia, la enciclopedia libre*. https://en.wikipedia.org/wiki/ANSI_escape_code

6. Enlaces del proyecto

A continuación se detallan los enlaces al código fuente del proyecto y al video explicativo.

Recurso	Enlace
Repositorio del proyecto	https://github.com/luzmrobledo/UTN-TUP-TPI-PROGRAMACION-1.git
Video explicativo	TPI-PROGRAMACION1.mp4